[J] Predicting coordinates of central carbon atoms in proteins from their amino acid sequence.

Andreas Munck
(Dated: April 30, 2026)

A protein's function is determined by its 3D structure. Determining this structure can help in understanding current proteins, designing new ones and curing diseases caused by errors in the structure. A problem is that experimentally determining these structures is a time-consuming and costly process. However, amino acid information is easier to obtain and this can then be used to predict the protein's structure. The aim of this project is to use amino acid sequences to predict the proteins central carbon atoms' coordinates. This was done with a model that predicted the distances between amino acids that built on a transformer part, multiple sequence alignment and triangular multiplicative update. A gradient descent-based method was then used to determine the coordinates from the distances. Two datasets were used, the root mean square error for the central carbon atoms, on the larger dataset, was 12.2 Å. Based on this result together with an analysis of the predicted structures, it was concluded that the model could not reliably predict the coordinates of central carbon atoms.

Project Topic: Predicting coordinates of central carbon atoms in proteins from their amino acid sequence.

Teaching Assistant: Aarón Domenzain

I. INTRODUCTION

Understanding protein functions is crucial for drug development and exploring mechanisms of life. A protein's function is determined by its 3D structure. However, determining this structure experimentally is a time-consuming and costly process. In contrast, determining a protein's amino acid sequence is much faster and cheaper. Machine learning models like AlphaFold 2 predict protein 3D structure from amino acid sequences with near-experimental accuracy [1]. However, AlphaFold 2 requires substantial computational resources for both training and inference. There are problems that require simpler models that predict faster, such as the problem of identifying antibodies in a patient's blood that can be relevant for a specific diagnosis. This is a hard problem for AlphaFold 2 because the human body can produce specific antibodies for molecules entering the body [2]. It is believed that the human body can make at least 10^{12} different antibody molecules [3]. This project investigates the possibility of predicting the position of one atom, the central carbon atom (C_α), in each amino acid of a protein. Further, the project aims to answer the question: How well can a neural network-based model predict the coordinates of a protein's central carbon atoms based on their amino acid sequence with limited computing capacity?

II. OVERVIEW

To predict C_α coordinates from amino acid (AA) sequences, a model needs to learn about the AAs and their positional interactions. A key challenge is therefore to teach a model as much as possible from sequential data. Three common approaches to do this are using recurrent connections, dilated convolutions or self-attention.

In the paper that introduces self-attention, the authors compare these three methods by their *complexity per layer* (CPL), the computational time complexity per layer, *minimum sequential operations* (MSO), the number of sequential operations needed to process the whole input sequence, and *maximum path length* (MPL), measuring the number of operations for one element to affect another distant element in the sequence [4].

Recurrent Connections

Recurrent networks include feedback connections so that previous outputs influence current computations [5]. This creates sequential dependencies that feed forward networks lacks. However, the feedback connections leads to longer training times, since the model only processes one element at a time, requiring $\mathcal{O}(n)$ MSO, where n is the sequence length. Furthermore, since information needs to travel element-by-element, the MPL is $\mathcal{O}(n)$. Finally, in each layer each element in the sequence is multiplied by a weight matrix of size $d \times d$. Therefore, the CPL is $\mathcal{O}(n \cdot d^2)$, where d is the hidden dimension.

Dilated Convolutions

Dilated convolutions insert spaces between kernel elements to widen the receptive fields without reducing resolution [6]. For standard convolutions with kernel width k, $\mathcal{O}(n/k)$ layers are needed to connect all elements in the sequence, if $k < n$. Consequently, the MPL becomes $\mathcal{O}(n/k)$. However, for a dilated kernel with the spaces between kernel elements growing exponentially with layers, the MPL is $\mathcal{O}(\log_k n)$. The MSO is $\mathcal{O}(1)$. The CPL is $\mathcal{O}(k \cdot n \cdot d^2)$, since for each element a convolution is computed over k elements and then multiplied with a $d \times d$ matrix. Dilated convolutions were used in the original AlphaFold 1 model [7].

Self-attention
The self-attention mechanism enables a MLP of $\mathcal{O}(1)$, introduced in the transformer model allowing sequence learning without recurrence and convolutions [4]. Each token in the sequences is represented by an embedding vector which elements are learned by the model. The self-attention mechanism does not take into account position in the sequence, therefore this also has to be encoded in a position vector. Self-attention creates three vectors for each token: queries (what information to seek), keys (what information to provide), and values (the actual information). Attention weights are computed by taking the dot-products between queries and all keys, determining how much each token should attend to others. This allows any token to directly influence any other token in one step, resulting in a MLP and MSO of $\mathcal{O}(1)$. These dot-products also leads to a CPL of $\mathcal{O}(n^2 \cdot d)$. This can however be reduced if each query only considers a neighbourhood of r tokens, reducing the context size of each embedding, to $\mathcal{O}(r \cdot n \cdot d)$.

Triangle Attention and Triangle Update
AlphaFold 2 introduces triangle multiplicative update and triangle attention [1]. These use a third amino acid as reference to try and introduce geometric constraints and learn three-way relations, such as the triangle inequality. The authors of AlphaFold2 points out that using only one of these methods resulted in good accuracy, however using both gave the best results [1]. Triangle multiplicative update modifies the relationship between AAs i and j using the product of the relations ik and jk for all $k \neq i, j$. There are two variants, in and out, since paired relations can depend on the direction, i.e., whether to use $i \rightarrow k$ or $i \leftarrow k$. Triangle attention computes attention over all relations ij with a reference of ik for all k. Both mechanisms have a CPL of $\mathcal{O}(n^3 \cdot d)$, due to the third reference point. Triangle multiplicative update only uses comparison by multiplication, it has a memory complexity of $\mathcal{O}(n^2 \cdot d)$, while triangle attention computes attention weights and therefore requires $\mathcal{O}(n^3 \cdot d)$ memory complexity.

Multiple Sequence Alignment
Multiple sequence alignment (MSA) extracts evolutionary information from AA sequences by aligning them with similar sequences [7]. MSA identifies positions in sequences which have conserved AAs and where they differ. For example, if one mutation changes the charge of an AA, a neighbouring AA may also mutate to compensate, indicating that these are spatially close. Large models such as AlphaFold 1 and 2 use large MSA databases to extract this information [1, 7].

III. METHOD

The model architecture was multi-stage. A transformer based network was used to learn paired relations between AAs, a MSA representation was learned by self-attention and then added to the paired representation. Further, triangle multiplicative update was used to form a *distogram*, a distance histogram. This was then converted to coordinates by a separate stage with no neural networks involved. An overview of the model is given in Figure 1. The machine learning python library PyTorch was used throughout the model. The models were trained on a Macbook M3 Pro with 18 GB of RAM. The models used a batch size of 128 and the Adam optimizer with a learning rate of 10^{-4}.

Data Collection
Protein structure IDs were obtained through the rcsb-api [8], which builds on the RCSB.org API [9]. Two datasets were gathered, by filtering based on the attributes shown in Table II.

TABLE II. **Attributes used to filter for suitable proteins.**

Attribute	Condition
# chains in molecule	= 1
Resolution	$\leq$ 3.5 Å
Polymer type	= "Protein"

The first and third attributes in Table II ensure that the protein is connected in one chain. The resolution was set to be less or equal to 3.5 Å to balance precise data and data size. Sequences containing more than 10 % unknown AAs were excluded from the dataset. The resulting two datasets, created by applying length limits to the chains can be seen in Table III.

TABLE III. **Datasets used**, distinguished by minimum (L_{min}) and maximum AA sequence length (L_{max}).

Dataset Label	L_{min}	L_{max}	#Samples
A	5	50	312
B	20	70	721

Dataset B's increased length range allowed for a larger dataset and more complex folded chains, as can be seen in Table III. The datasets were split with 90 % to training/validation and 10 % to the testing, with the training/validation set being split with 80 % to training.

Encoding Data
Learned embeddings encoded amino acid sequences, with sequences shorter than the maximum protein length being padded with zeros. Sinusoidal position encoding of the sequence was then added to these embeddings.

Directly predict coordinates from AA sequences lacks invariance to global rotations and translations. Therefore, the C_α-coordinates were converted to Euclidean distance matrices. However, even the relative distances fluctuate due to thermal fluctuations, amongst other reasons. Therefore, the model instead predicts a probability

TABLE I. **Overview of Machine Learning Methods for Protein Structure Prediction.** This table compares different neural network architectures and mechanisms for learning from amino acid sequences and predicting 3D protein coordinates. Methods are evaluated based on their computational complexity, ability to capture long-range dependencies, and suitability for projects with limited computational resources.

Method	**Use case scenario**	**Features**	**Suitable for the project?**
Recurrent Connection	Learning sequential dependencies in AA sequences	Uses a feedback connection for sequential dependencies. It has a MSO of $\mathcal{O}(n)$, MPL of $\mathcal{O}(n)$ and CPL of $\mathcal{O}(n \cdot d^2)$.	Recurrent connections are inferior to the following methods in this project due its ability of handling long sequences.
Dilated Convolutions	Learning sequential dependencies in AA sequences	Can be computationally efficient depending on n. It has a MPL of $\mathcal{O}(\log_k n)$, MSO of $\mathcal{O}(1)$, CPL $\mathcal{O}(k \cdot n \cdot d^2)$.	Inferior to self-attention due to its MPL. However, it can be a back-up option if self-attention turns out too computationally expensive on the hardware used.
Self-attention	Learning sequential dependencies in AA sequences	Learns longer range dependencies most efficiently. MPL of $\mathcal{O}(1)$, MSO of $\mathcal{O}(1)$ and CPL of $\mathcal{O}(n \cdot d^2)$. Can become costly computationally and memory-wise as n becomes large.	For a small enough maximum protein length n the self-attention mechanism should outperform the two methods above.
Triangular Multiplicative Update	Learning three-way relations	Learns three-way relations by comparing a relation between a pair of AAs to a relations to a third reference AA. It has a CPL $\mathcal{O}(n^3 d)$ and memory complexity of $\mathcal{O}(n^2 d)$	Suitable to learn three-way relations and uses less memory than triangular attention.
Triangular Attention	Learning three-way relations	Learns three-way relations by calculating attention for each pair of AAs with a third reference AA. It has a CPL of $\mathcal{O}(n^3 d)$ and memory complexity of $\mathcal{O}(n^3 d)$	Uses too much memory. The authors of Alphafold 2 stated that good accuracy can be achieved by just one of triangular update or attention [1], therefore only the triangular update is used.

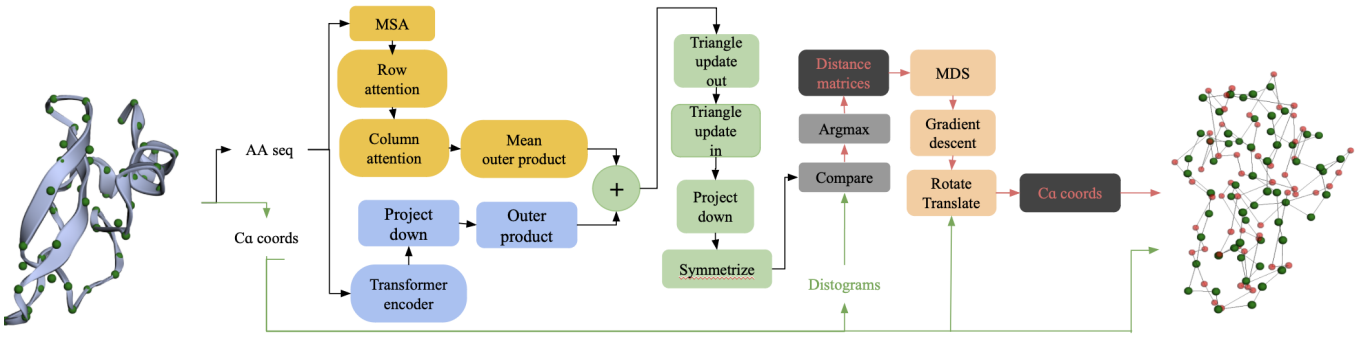

FIG. 1. **Overview of Model Used**. AA sequences were used as inputs to predict C_α coordinates. Information from the AA sequences was learned directly through the blue blocks. In the yellow block the information from similar sequences was used to better understand the structure. These two representations were further trained together in the green blocks. Finally in the orange blocks the predicted distances were converted to 3D coordinates.

distribution of the possible distances between AAs instead of predicting distances with regression. One instance of the model was trained on distance matrices, while two were trained on distrograms. The bin edges, in Å, were [0.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0, 7.0, 8.0, 9.0, 10.5, 13.0, 15.0, 17.0, 21.0, 25.0], forming 16 bins.

Matrix dimensions matched the maximum protein length in each dataset. A method to decrease the size of these and therefore the computational cost and memory usage in the attention mechanism would be to only consider $r <$ (max length), neighbouring AAs. On the one hand, this would have enabled the use of larger proteins as training data. On the other hand, proteins are often folded and AAs that are close spatially are not necessarily close in the amino acid chain.

MSA matrices were formed by aligning each AA sequence with the nine other most similar sequences in the dataset.

Learning from Sequences

The MSA matrices were first run through column-wise and then row-wise attention to limit the memory and computational complexity. The embedding dimension was 16 for dataset B and 32 for dataset A. Further, the MSA matrices were fed through two parallel linear layers, then the mean along the columns was taken followed by an outer product of these two rows.

The encoded AA sequences were fed to a transformer encoder, with 8 attention heads, 10 % dropout rate and embedding dimension 256. Dataset B used two transformer layers while dataset A used four. A linear layer was used to project down the dimension before taking the outer product, to match the dimension of the MSA.

After summing these matrix representations, triangle multiplicative update was applied, followed with a projection down to the number of bins, or to one distance in the case of regression. Finally, matrices were symmetrized by averaging the upper and lower triangles of the matrices before loss computation.

Distances to Coordinates

Classical multidimensional scaling (MDS) was first used to approximate the coordinates that best fit the distance matrix. MDS transforms the square of the distance matrix so that the mean of all rows and columns are zero. Then it calculates the coordinates from the eigenvalues and eigenvectors. To refine the coordinates, gradient descent with LBFGS, was applied to the loss function:

$$L = \sum_{i=1}^{N} \sum_{j=1}^{N} \frac{1}{2} (D_{ij} - ||\mathbf{X}_i - \mathbf{X}_j||)^2, \tag{1}$$

where $\mathbf{X}$ are the coordinates, D_{ij} the elements of the distance matrix $\mathbf{D}$. This yielded coordinates could be rotated, mirrored and translated to fit the correct coordinates.

IV. RESULTS AND DISCUSSION

The algorithm to convert distance matrices to coordinates almost works. Correct distance matrices from dataset B were used as a test. The root mean squared error (RMSE) after gradient descent and before and after rotation and translation is shown in Table IV.

TABLE IV. **Test of realizing coordinates from distance matrix.** Here $\mathbf{D}$ is the distance matrix and $\mathbf{X}_b$ are the coordinates before rotation and translation, while $\mathbf{X}_a$ is afterwards.

RMSE $\mathbf{D}$ [Å]	RMSE $\mathbf{X}_b$ [Å]	RMSE $\mathbf{X}_a$ [Å]
$1.30 \cdot 10^{-4}$	19.9	3.8

The 3.8 Å error after alignment is probably due to the coordinates being a mirrored. Most samples matched the original coordinates exactly. A protein does not necessarily retain its function when mirrored. However, this can be solved by adding constraints through, for example, also predicting angles. The problem is now almost reduced to finding a correct distance matrix. The results on the test sets for the predicted distance matrix and coordinates are shown in Table V.

TABLE V. **Test set results.** First the accuracy of predicting the correct bins (Acc) is shown. This is followed by the root mean square error (RMSE) in different stages of the model. $\mathbf{D}$ are the distance matrices and $\mathbf{X}_b$ are the coordinates before rotation and translation, while $\mathbf{X}_a$ is afterwards.

Dataset	Acc [%]	$\mathbf{D}$ [Å]	$\mathbf{X}_b$ [Å]	$\mathbf{X}_a$ [Å]
A	Regression	4.90	12.3	7.30
A	21.6	6.44	12.8	7.43
B	35.6	8.13	19.9	12.2

Note that the two classification model instances were not tuned as thoroughly as the instance for regression. Dataset A also gives better results, partly because shorter proteins are on average smaller, but probably mostly because a lot of proteins in data set A did not fold. Therefore, the model easily learned to predict proteins that were further away in the sequence to be further away in 3D space. The model was not drastically of in predicting some small proteins, but had a harder time with larger ones. This is illustrated in Figure 2.

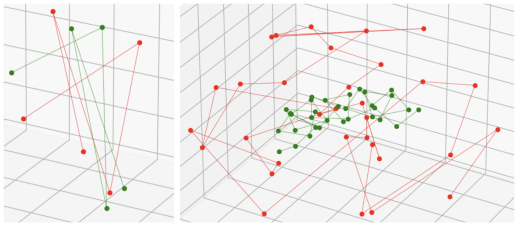

FIG. 2. **Sample from predictions of dataset A**. To the left a shorter AA sequence. To the right a longer AA sequence with more folding. In green are the correct coordinates and in red the predicted. The dataset A was used here and the model was the classification instance.

The model trained on dataset B probably tried to learn the folding more, as seen in Figure 3. This is probably due to the larger and higher-quality dataset.

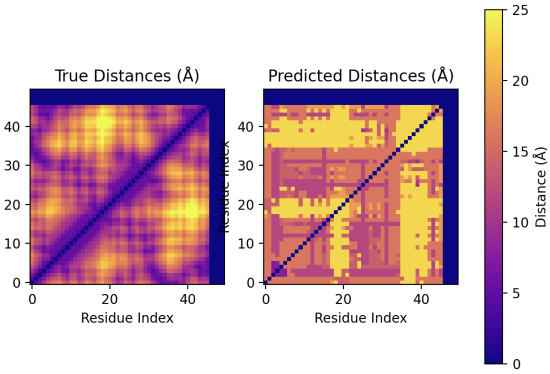

FIG. 3. **Distance matrix from prediction on dataset B.** RMSE for this protein was 7.04 Å and the bin accuracy 29 %. Correct distances between residues, another name for amino acids, to the left and predicted distances to the right. The colorbar gives the distances in Å.

Some predictions on long sequences in dataset B were quite close, while others were far off. One example where it was close is shown in Figure 4.

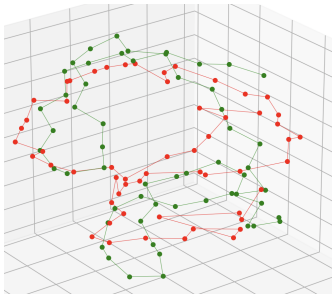

FIG. 4. **Prediction of one sample from dataset B**. The model used was trained on dataset B. Correct coordinates in green and predicted in red.

V. CONCLUSIONS AND OUTLOOK

The neural network-based models used here could not predict the 3D structure of proteins. The primary reason for this is probably too small of a dataset with too short sequences. However, this project demonstrates that coordinate prediction can largely be reduced to distance prediction between amino acids, except for the risk of obtaining a mirrored structure.

For future work, if using limited computational resources, a promising approach could be to replace the MSA with a model pre-trained on a large amount of sequences. To simplify, a future project could be to try and predict atoms that are in contact, i.e. closer than 8 Å.

VI. CONTRIBUTIONS

A.M. implemented the full model and wrote the full report.

VII. CONFLICT OF INTEREST

The author is not aware of having any conflicts of interests.

VIII. DATA AND CODE AVAILABILITY

The data and code for this project are made publicaly available on GitHub at `https://github.com/pAndreasM/AML-Project-J`.

[1] J. Jumper, R. Evans, *et al.*, Highly accurate protein structure prediction with alphafold, Nature **596**, 583–589 (2021), relevant because it introduces Alphafold 2, that can achieve near-experimental accuracy in protein 3D structure prediction.

[2] B. Briney, A. Inderbitzin, *et al.*, Commonality despite exceptional diversity in the baseline human antibody repertoire, Nature **566**, 393–397 (2019), relevant because it highligts a problem where a faster prediction model is needed.

[3] B. Alberts, A. Johnson, *et al.*, The generation of antibody diversity, Molecular Biology of the Cell, 4th Edition, Garland Science **Chapter 24**, Section 24 (2002), relevant because it highligts a problem where a faster prediction model is needed.

[4] A. Vaswani, N. Shazeer, *et al.*, Attention is all you need, Advances in Neural Information Processing Systems **30**, 5998 (2017), relevant because it introduces the transformer, a useful architecture that uses the attention mechanism for capturing long range dependencies in sequences.

[5] S. Hochreiter and J. Schmidhuber, Long short-term memory, Neural Computation **9**, 1735 (1997), relevant because it introduces LSTM and describes RNNs, predecessor to transformers in learning from sequences.

[6] F. Yu and V. Koltun, Multi-scale context aggregation by dilated convolutions, arXiv preprint arXiv:1511.07122 (2015), relevant because it introduces dilated convolutions.

[7] A. W. Senior, R. Evans, J. Jumper, *et al.*, Improved protein structure prediction using potentials from deep learning, Nature **577**, 706–710 (2020), relevant because it introduces Alphafold 1, able to predict 3D structures from amino acid sequences.

[8] D. W. Piehl, B. Vallat, *et al.*, rcsb-api: Python toolkit for streamlining access to rcsb protein data bank apis, Journal of Molecular Biology https://doi.org/10.1016/j.jmb.2025.168970 (2025), used for accessing protein data in this project.

[9] Y. Rose, J. M. Duarte, *et al.*, Rcsb protein data bank: Architectural advances towards integrated searching and efficient access to macromolecular structure data from the pdb archive, Journal of Molecular Biology https://doi.org/10.1016/j.jmb.2020.11.003 (2020), describes the API services used by rcsb-api.